Fouad Aladhami
210102002808

MİCROPROCESSORS HM3

1) Problem 1

a) Assembly code

To calculate the Hamming weight of the 32-bit word stored in R1 register the following algorithm can be used. In R2 the total number of need to be checked will stored, here it is 32 for 32-bit word. The word is shifted to the right if 1 pattern appear in the LSB the carry flag will be set so the value of the hamming weight can be increased and count down, else the hamming weight will not increase just continue to count down when checking bits.

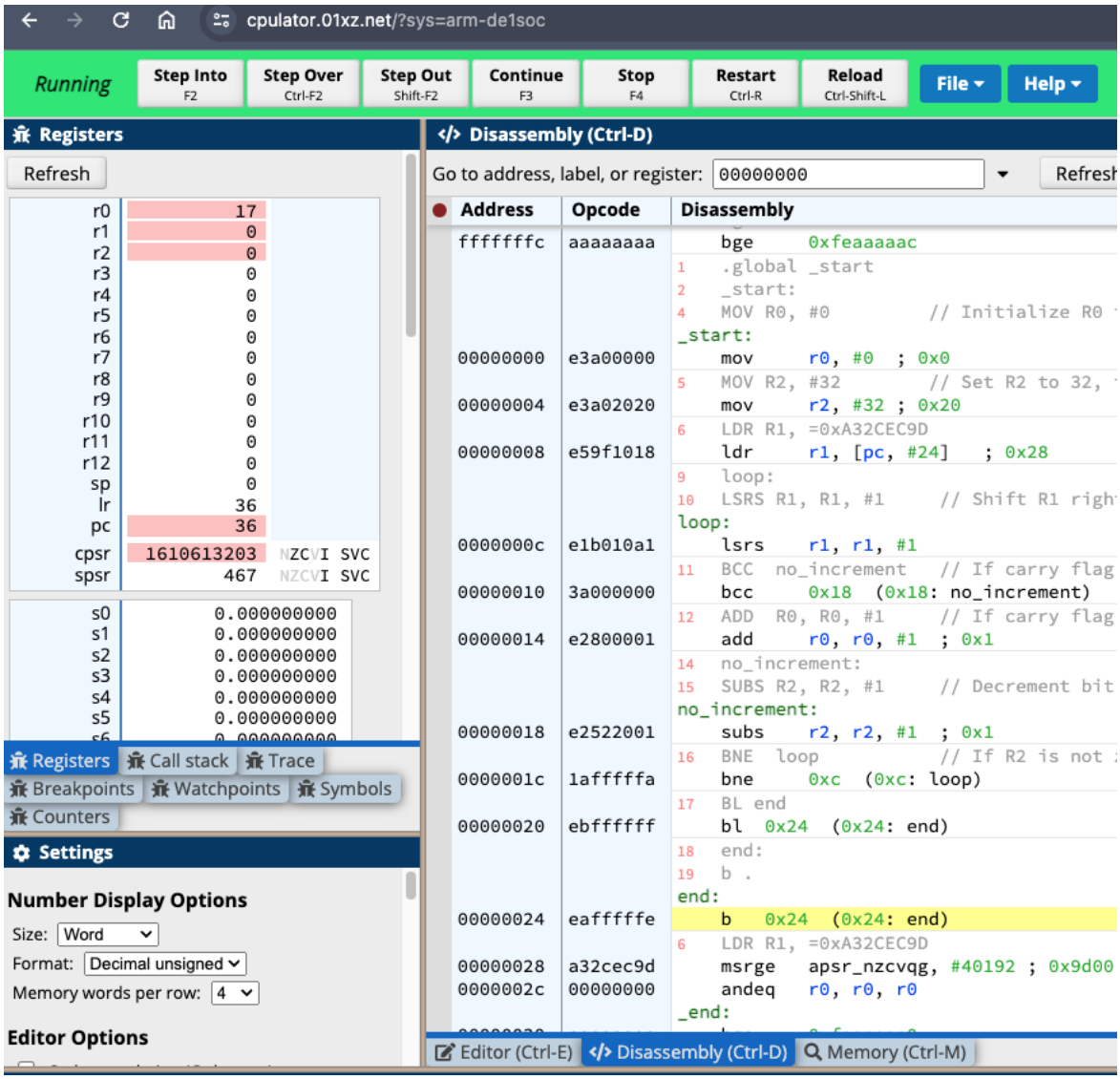
The code is as following:

```
.global _start
_start:
   MOV R0, #0        // Initialize R0 to 0, which will store the result
   MOV R2, #32       // Set R2 to 32, the number of bits to check
   LDR R1, =0xA32CEC9D


loop:
   LSRS R1, R1, #1     // Shift R1 right by 1 bit, store result in R1
   BCC  no_increment   // If carry flag is clear, bit was 0, jump to no_increment
   ADD  R0, R0, #1     // If carry flag is set, bit was 1, increment R0

no_increment:
   SUBS R2, R2, #1     // Decrement bit count
   BNE  loop          // If R2 is not zero, repeat the loop
   BL end
end:
          b .
   // At this point, R0 contains the Hamming weight
```

The code is tested in cpulator onlline assembly simulator and the solution for the given example is finded as 17 and its true. Because R1 is equal to 0xA32CEC9D at the beginning And its equal to 10100011001011001110110010011101 in binary. It’s include 17 “1” s.



b) C code

To calculate hamming weight in C a similar approach is used. But because it's not possible to use carry flag the value is shifted to right and the LSB is added to the count variable in each time to increase the hamming value.

The code with explanation is as following:

```
#include <stdio.h>
#include <stdint.h>

uint32_t hammingWeight(uint32_t n)
{
   uint32_t count = 0;
   while (n)
   {
      count += n & 1; // Add the least significant bit of n to count
```

```
        n >>= 1;       // Right shift n by 1 bit
    }
    return count;
}

int main()
{
    uint32_t number = 0xA32CEC9D; // Example number
    printf("Hamming Weight of %X is %u\n", number, hammingWeight(number));
    return 0;
}
```

When the c code executed in vs code the following output is produced.

```
fuad@Fouad-MacBook-Air output % ./"main"
Hamming Weight of A32CEC9D is 17
```

2) Problem 2

To find the hamming distance, the number of different bits in the two given number must be counted. XOR operation of two number generate “1” bit for each different bit. So, if the two number XORed than using the same Hamming weight algorithm, the hamming distance can be funded. Because every bit equal to “1” in the XORed variable mean a bit differ.

a) Assembly code

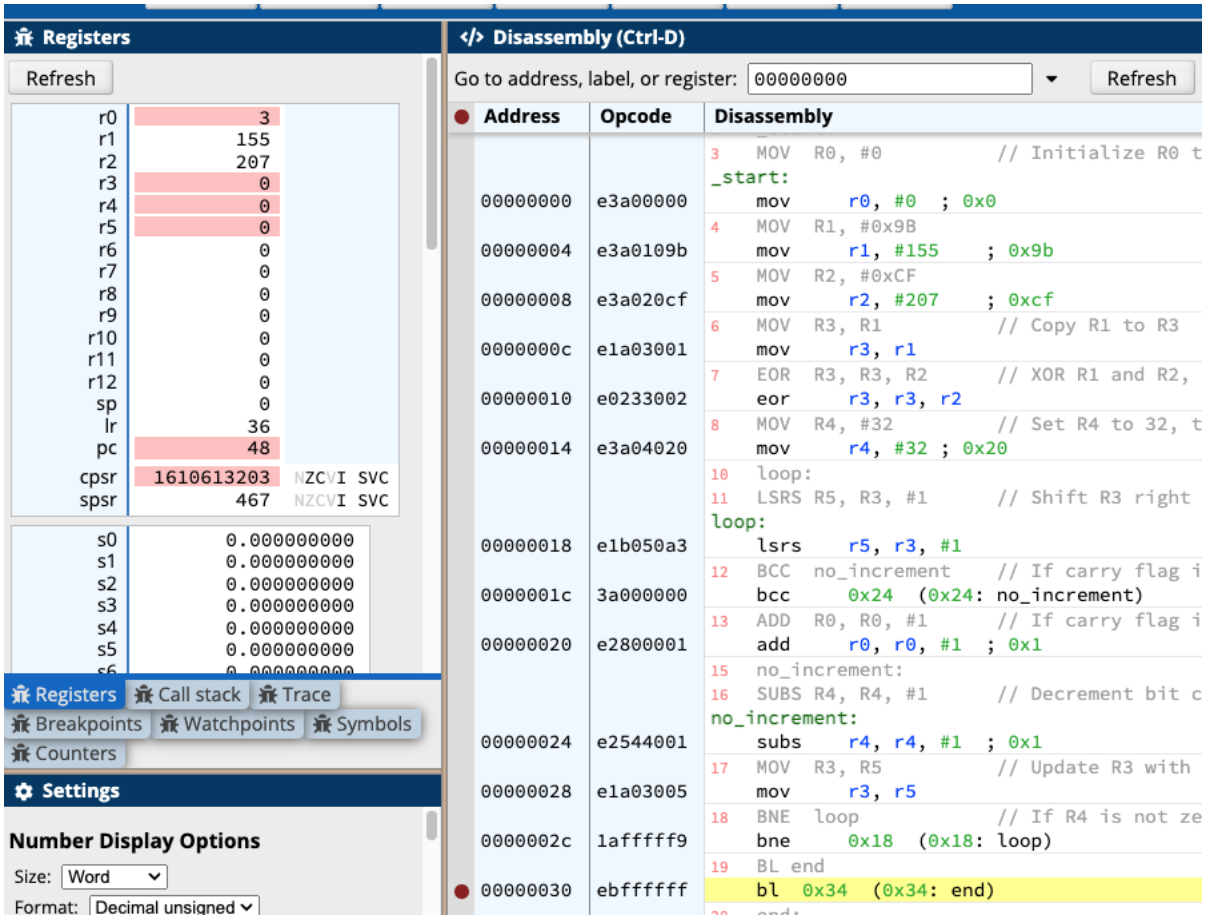
```
.global _start
_start:
        MOV  R0, #0        // Initialize R0 to 0, which will store the result
        MOV    R1, #0x9B              // 10110101 in binary this is the first number
        MOV    R2, #0xCF              // 11001111 in binary this is the second number
   MOV  R3, R1        // Copy R1 to R3
   EOR  R3, R3, R2     // XOR R1 and R2, store the result in R3
   MOV  R4, #32        // Set R4 to 32, the number of bits to check

loop:
   LSRS R5, R3, #1     // Shift R3 right by 1 bit, store result in R5
   BCC  no_increment    // If carry flag is clear, bit was 0, jump to no_increment
   ADD  R0, R0, #1     // If carry flag is set, bit was 1, increment R0

no_increment:
   SUBS R4, R4, #1     // Decrement bit count
   MOV  R3, R5        // Update R3 with the shifted value
   BNE  loop          // If R4 is not zero, repeat the loop
        BL end
end:
        b . // At this point, R0 contains the Hamming distance
```

Fouad Aladhami
210102002808

The code is tested in cpulator online emulator, and the result is as following. When the code execution completed it is clearly seen that R0 register value is equal to 3. And its work successfully.



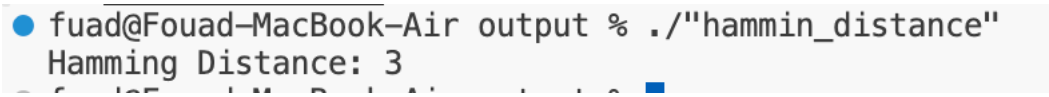
b) C code

```
#include <stdio.h>
#include <stdint.h>

uint32_t hammingDistance(uint32_t a, uint32_t b)
{
   uint32_t xor = a ^ b; // XOR to find differing bits
   uint32_t count = 0;
   while (xor)
   {
      count += xor&1; // Count the number of 1s
      xor >>= 1;      // Shift right by 1
   }
   return count;
}
```

```
int main()
{
   uint32_t a = 0x9B; // Example: 10110101 in binary
   uint32_t b = 0xCF; // Example: 11001111 in binary
   printf("Hamming Distance: %u\n", hammingDistance(a, b));
   return 0;
}
```

When the code is executed in vs code the output of the code is as following.



The code work successfully and find the hamming distance.